

TALLER ANALÍTICO 1: INDEXACIÓN Y TENSORES (15 MIN)

Respondan las siguientes preguntas teóricas discutiendo en grupo:

1. En la Matriz A de arriba, ¿cuál es el valor exacto del elemento $A_{2,3}$ (Asumiendo índice base 0 para i y j)? Justifique qué representa visualmente.
2. Si tenemos un Tensor RGB de dimensiones $1080 \times 1920 \times 3$, ¿cuántos valores individuales (bytes) se están almacenando en memoria para esa única imagen sin comprimir? Realice el cálculo.

1. Para resolverlo, primero ubicamos la fila 2 y la columna 3 usando el conteo desde el 0. Si miramos la fila 2, que es $[255, 0, 128, 0, 255]$, al contar las posiciones (0, 1, 2, 3), el valor que encontramos es **0**. Visualmente, ese 0 representa el color negro, es decir, ausencia total de luz. En este caso, ese punto exacto de la imagen forma parte de las líneas oscuras que rodean al pequeño cuadro gris de valor 128 que está justo en el centro.
2. Multiplicar los tres números: $1080 \times 1920 \times 3 = 6.220.800$ valores (bytes). Es decir, para guardar una sola foto sin comprimir en el computador, se necesita almacenar más de 6 millones de números

TALLER DE LABORATORIO 1: TRANSFORMACIONES AFINES (20 MIN)

Reto en Python: Imaginen que trabajan en software para radiografías. Las imágenes están sobreexpuestas (casi completamente blancas).

1. Abran su IDE de Python. Creen una matriz de prueba 5x5 usando `np.random.randint(200, 255, (5, 5))`.
2. Apliquen matemáticamente una reducción de contraste del 50% y disminuyan el brillo en 50 unidades.
3. Asegúrense de usar `np.clip()` y convertir a enteros `np.uint8`.
4. Impriman la matriz original y la procesada para comparar.

The screenshot shows a VS Code editor with a file named `indexacion.py` open. The code in the editor is as follows:

```
1 import numpy as np
2 matriz_original = np.random.randint(200, 255, (5, 5))
3
4 alpha = 0.5
5 beta = -50
6
7 matriz_nueva = (alpha * matriz_original) + beta
8
9 matriz_procesada = np.clip(matriz_nueva, 0, 255).astype(np.uint8)
10
11 print("Matriz original:\n", matriz_original)
12 print("Matriz procesada:\n", matriz_procesada)
```

Below the editor, the terminal window shows the execution of the script. The output is as follows:

```
@gomezzuley19-debug →/workspaces/IA-2_Talleres (main) $ source /workspaces/IA-2_Talleres/.env/bin/activate
(.venv) @gomezzuley19-debug →/workspaces/IA-2_Talleres (main) $ /workspaces/IA-2_Talleres/.env/bin/python "/workspaces/IA-2_Talleres/Taller 1/indexacion.py"
Matriz original:
[[237 239 233 253 248]
 [231 245 221 251 245]
 [210 230 232 232 214]
 [244 243 221 246 236]
 [251 234 231 211 223]]

Matriz procesada:
[[68 69 66 76 74]
 [65 72 60 75 72]
 [55 65 66 66 57]
 [72 71 60 73 68]
 [75 67 65 55 61]]
(.venv) @gomezzuley19-debug →/workspaces/IA-2_Talleres (main) $
```

TALLER ANALÍTICO 2: TRANSFORMACIONES (15 MIN)

1. Si transponemos una matriz identidad I de 4×4 , ¿cuál es la matriz resultante?
Explique geoméricamente por qué.
2. Una imagen RGB tiene una forma de $(200, 200, 3)$. Si la "aplanamos" para introducirla en la capa de entrada de una Red Neuronal, ¿cuántas neuronas exactas deberá tener esa capa de entrada para recibir el vector?

1. Resulta la misma matriz identidad, sin ningún cambio. Porque la matriz identidad solo tiene 1s en la diagonal (de esquina a esquina) y 0s en todo lo demás. Transponer es "voltear" la matriz sobre esa diagonal, pero como la diagonal se queda igual (los 1s no se mueven), el resultado final es idéntico al original. Es como voltear una imagen simétrica: se ve exactamente igual
2. $200 \times 200 \times 3 = 120.000$ neuronas. Esto es porque "aplanar" significa poner todos los valores de la imagen (alto, ancho y los 3 colores) en una sola fila larga, uno detrás de otro. Cada número necesita su propia "entrada" en la red neuronal

TALLER DE LABORATORIO FINAL: PROGRAMANDO UN KERNEL (25 MIN)

Misión Práctica: Construyan un simulador básico de convolución para una matriz de 3x3 (sin usar librerías avanzadas como OpenCV, solo matemáticas en Python/NumPy).

1. Creen en NumPy la matriz "Sección de Imagen (I)" y el "Kernel (K)" mostrados en el diagrama superior.
2. Escriban el código para calcular el valor central resultante si superponemos el Kernel sobre la Imagen. *(Pista: Deben multiplicar la matriz 'I' por la matriz 'K' usando el producto Hadamard y luego sumar TODOS los valores de la matriz resultante usando `np.sum()`).*
3. Impriman el valor del píxel central calculado. Este es el principio fundamental de cómo un filtro detecta texturas en Deep Learning.

```

indexacion.py M X Laboratorio_Final_Kernel.py U X
Taller 1 > Laboratorio_Final_Kernel.py > ...
1 import numpy as np
2 I = np.array([
3     [100, 100, 100],
4     [100, 200, 100],
5     [100, 100, 100]
6 ])
7
8 K = np.array([
9     [0, -1, 0],
10    [-1, 5, -1],
11    [0, -1, 0]
12 ])
13
14 print("Sección de Imagen (I):")
15 print(I)
16
17 print("\nKernel (K):")
18 print(K)
19
20 producto_hadamard = I * K
21
22 print("\nProducto Hadamard (I * K):")
23 print(producto_hadamard)
24
25 pixel_central = np.sum(producto_hadamard)
26
27 print("\nValor del píxel central calculado:", pixel_central)

```

(.venv) @gomezzuley19-debug → /workspaces/IA-2_-Talleres (main) \$ /workspaces/
 inal_Kernel.py"
 Sección de Imagen (I):
 [[100 100 100]
 [100 200 100]
 [100 100 100]]

Kernel (K):
 [[0 -1 0]
 [-1 5 -1]
 [0 -1 0]]

Producto Hadamard (I * K):
 [[0 -100 0]
 [-100 1000 -100]
 [0 -100 0]]

Valor del píxel central calculado: 600

(.venv) @gomezzuley19-debug → /workspaces/IA-2_-Talleres (main) \$ █